# APP 3D
# COMPUTER GRAPHICS

Bartocetti Enrico

Ingegneria e Scienze Informatiche – A.A. 2025/2026

# ASPETTI FUNZIONALI

## Sommario


1. Menu principale
2. Tema scelto
3. Navigazione e interazione
4. Modifica oggetti
5. Modifica mesh
6. Caricamento di file
7. Illuminazione
8. Collisioni
9. Effetti visivi
10. Comandi Utente

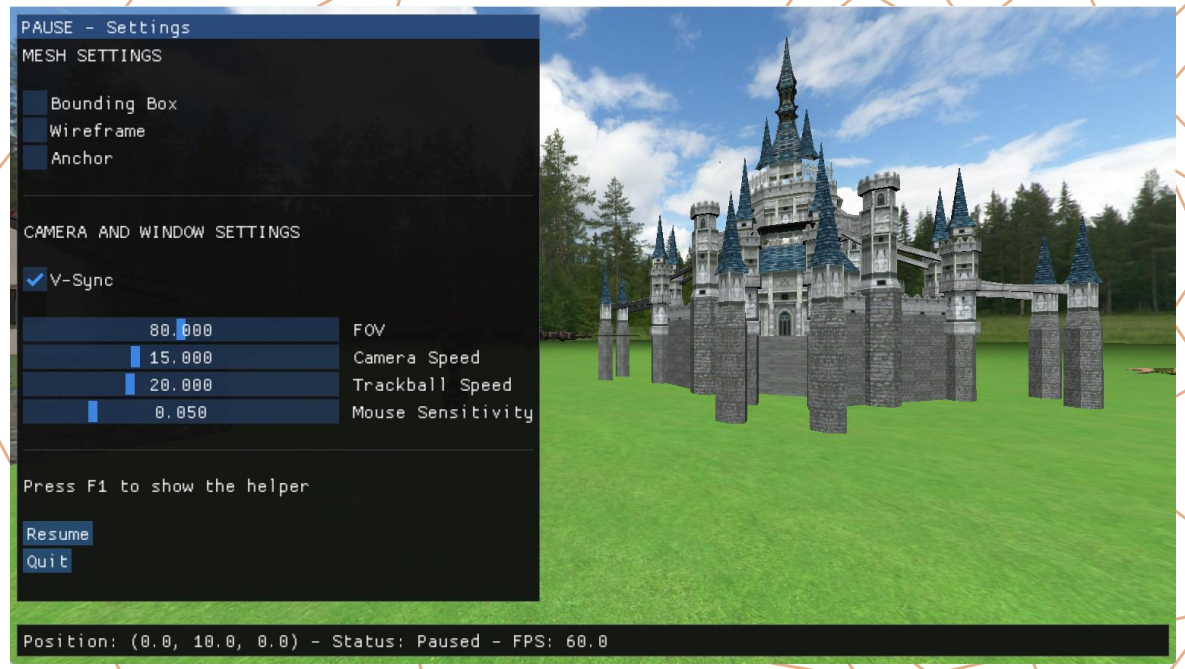

Successivamente:
Progettazione del Software

# MENU PRINCIPALE

Il Menu Principale viene visualizzato all'avvio dell'applicazione e alla pressione del **tasto P**.

In questa schermata è possibile **modificare** varie **impostazioni,** tra cui:

- **Rendering** di bounding box, ancore e wireframe
- **Campo** visivo della telecamera
- **Velocità** di movimento tramite WASD e di trascinamento tramite trackball
- **Sensibilità** del mouse nei movimenti
- Abilitazione **V-Sync**

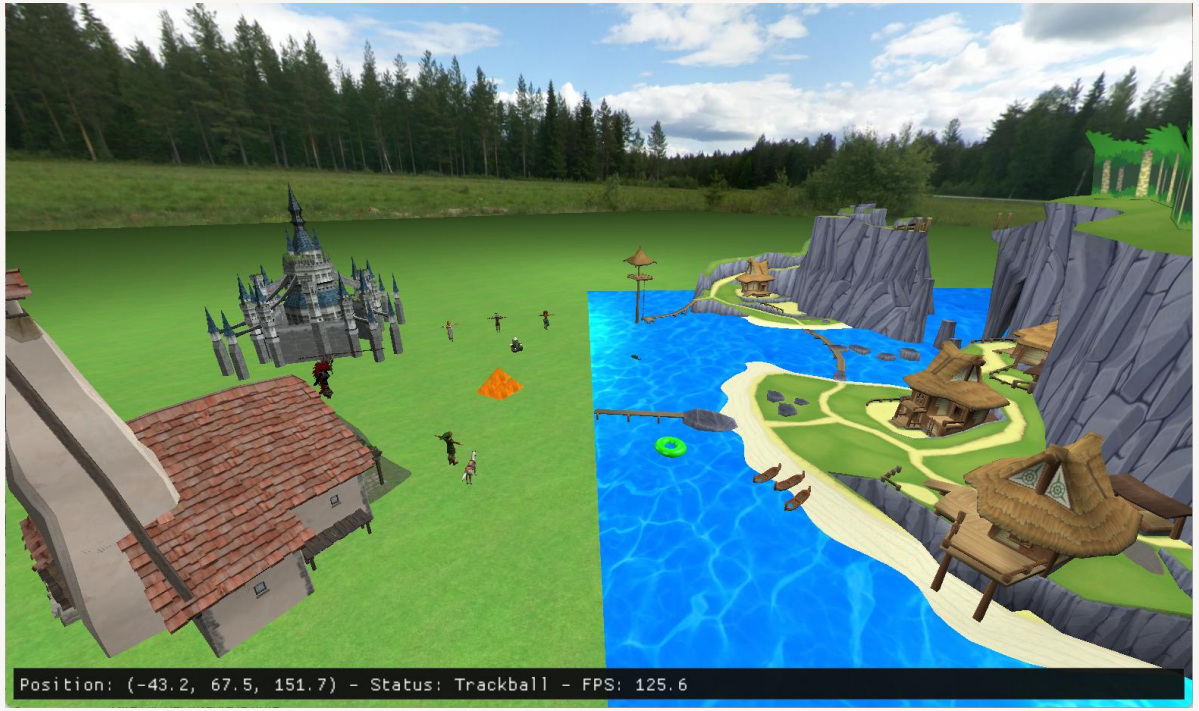


# TEMA SCELTO

L'applicazione rappresenta una scena del gioco **The Legend of Zelda**, contenente:

- Vari personaggi principali
- Casa di Link e Castello di Hyrule
- Isola Primula, con l'acqua animata a livello di shader
- Piramide rotante della Triforza

Tutti i file mesh importati sono stati **scaricati** dal sito [The Models Resource] ed esportati in .obj tramite Blender, eccetto la triforza modellata tramite piramidi, il salvagente tramite toro, il piano d'appoggio e il mare tramite piani.



# NAVIGAZIONE E INTERAZIONE

Sono disponibili 3 modalità per **interagire** con la scena:

- Navigazione
- Trackball
- Picking

È possibile cambiare modalità utilizzando la **barra spaziatrice**: a ogni pressione si passerà alla modalità successiva.

In qualsiasi momento si possono comunque utilizzare le **frecce direzionali** per direzionare la telecamera e i tasti **WASD** per spostarla.

Quando in altri stati, per tornare alla precedente modalità di interazione è necessario utilizzare ripetutamente il tasto ESC.

## NAVIGAZIONE

In modalità navigazione il cursore viene catturato dall'app, così che il **movimento del mouse** corrisponda allo spostamento della telecamera, come accade nei videogiochi in prima persona.

# NAVIGAZIONE E INTERAZIONE

## TRACKBALL

In questa modalità il cursore è visibile poiché viene simulata una **sfera** contenente il mondo che l'utente può afferrare con il pulsante sinistro del mouse e **trascinare** nelle varie direzioni (passando nello stato trackball moving).

```
Position: (-3.0, 10.2, -1.8) - Status: Trackball - FPS: 115.4
```

```
Position: (-3.0, 10.2, -1.8) - Status: Trackball moving - FPS: 111.6
```

## PICKING

Quando si è in picking l'utente può cliccare sulle **ancore** delle mesh (visualizzabili tramite flag nel menu di pausa) per aprirne il menu di modifica.

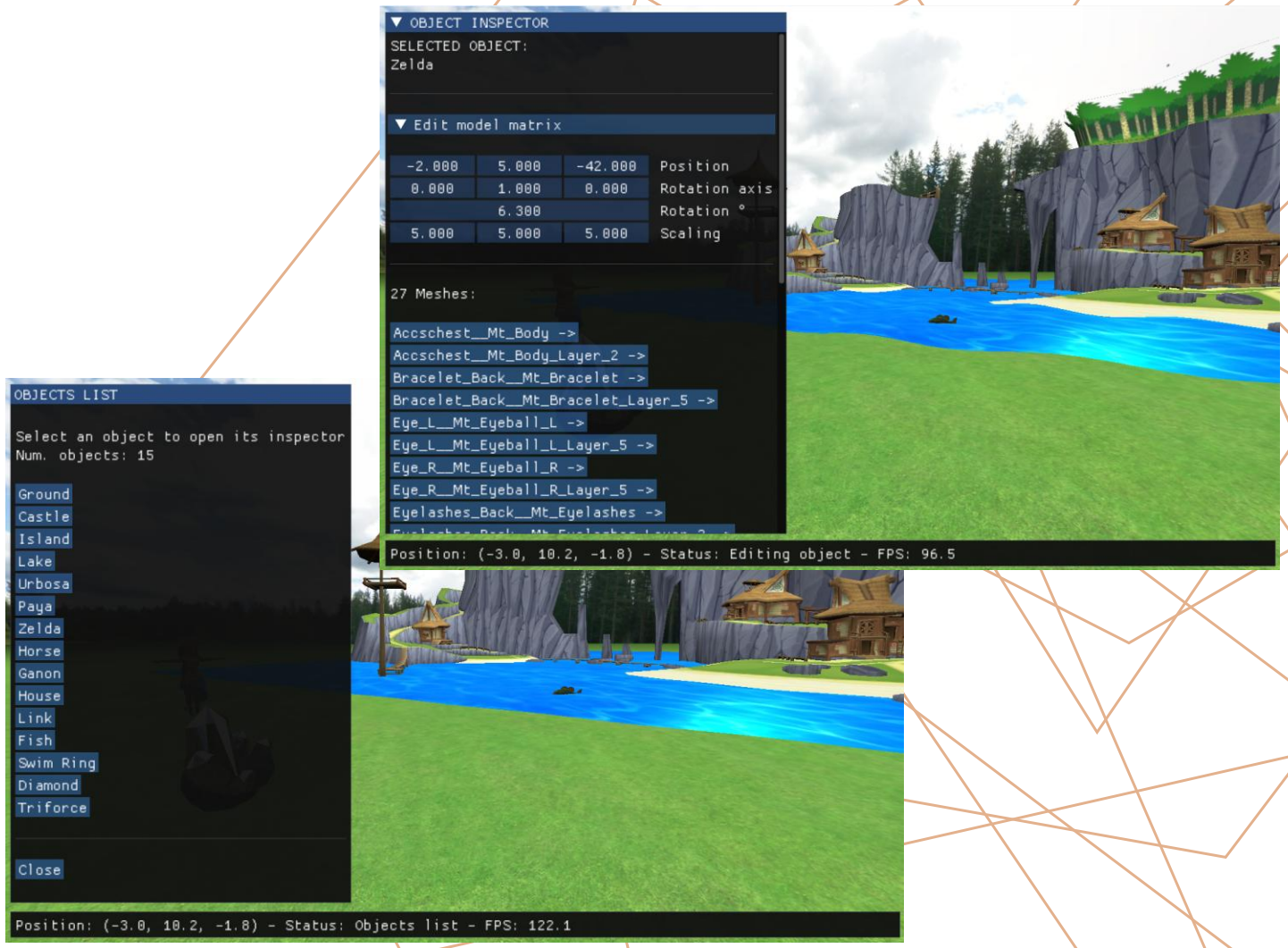
```
Position: (-3.0, 10.2, -1.8) - Status: Picking objects - FPS: 116.2
```

# MODIFICA OGGETTI

Poiché gli oggetti sono composti da più mesh, il menu di modifica degli oggetti è separato da quello delle mesh.

Premendo il **tasto O** si apre la **lista** di tutti gli oggetti presenti nella scena, permettendo di scegliere l'oggetto da modificare.

Una volta aperto il dettaglio di un oggetto, è possibile modificarne la **matrice di modellazione**, selezionare una delle **mesh che lo compongono** oppure **eliminarlo**.



# MODIFICA MESH

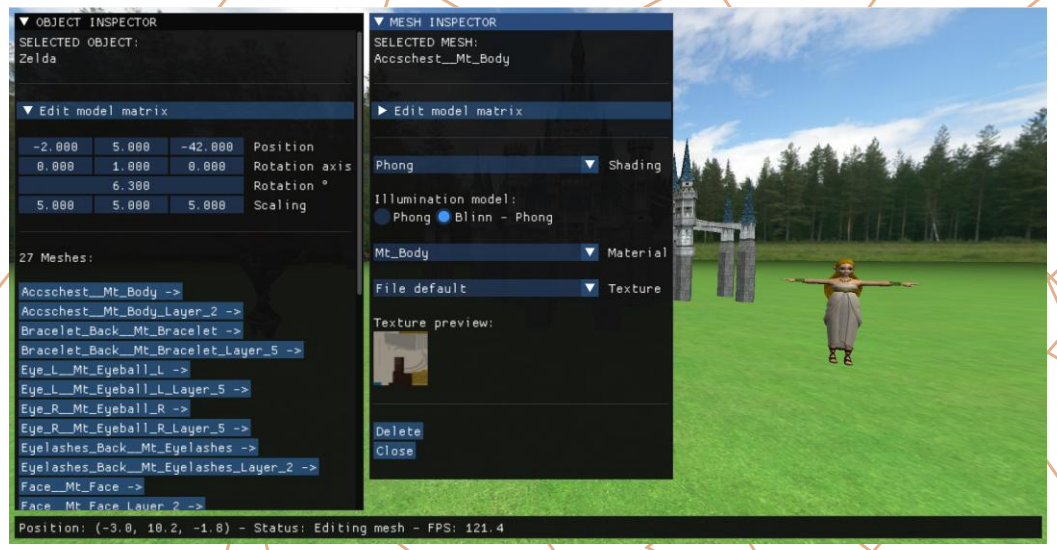
Il menu di modifica delle mesh viene aperto al click su un'ancora in modalità picking o tramite la selezione della stessa dalla lista delle mesh di un oggetto, permettendo di modificare:

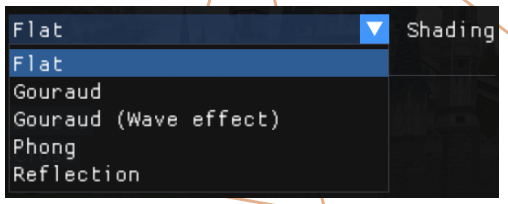
- **Matrice** di modellazione
- **Shader** da applicare
- **Rimuovere** la mesh dall'oggetto

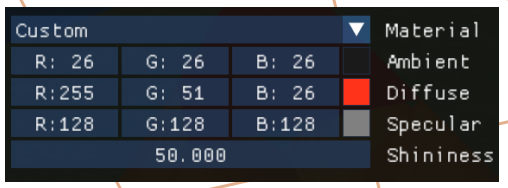
Inoltre, solo se lo shader selezionato lo permette (analizzando le uniform presenti):

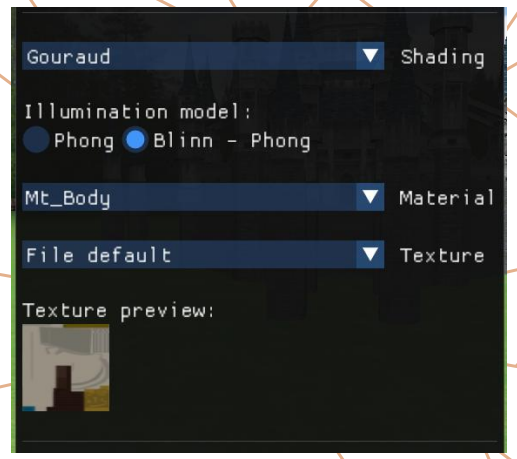
- Modello di **illuminazione** (Phong – Blinn Phong)
- **Materiale** (eventualmente personalizzato)
- **Texture** e relativa preview

Se la mesh è stata caricata da file, vengono anche proposti il **materiale** e la **texture importati**.



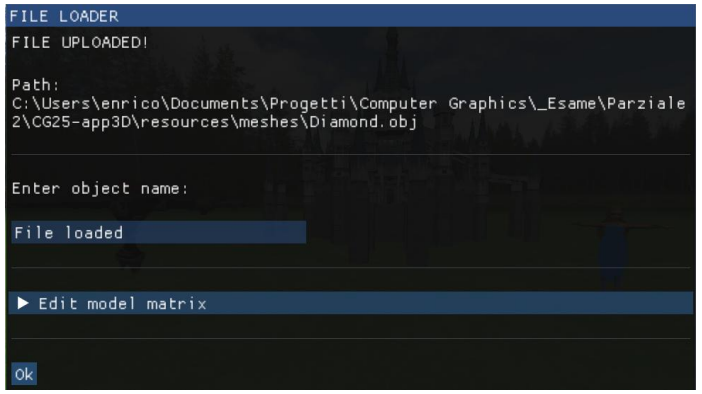



# CARICAMENTO DI FILE

Quando ci si trova in una delle **modalità di interazione**, è possibile **trascinare dei file** nell'app per eseguirne il caricamento. Sono riconosciuti file contenenti mesh e file immagini per le texture.

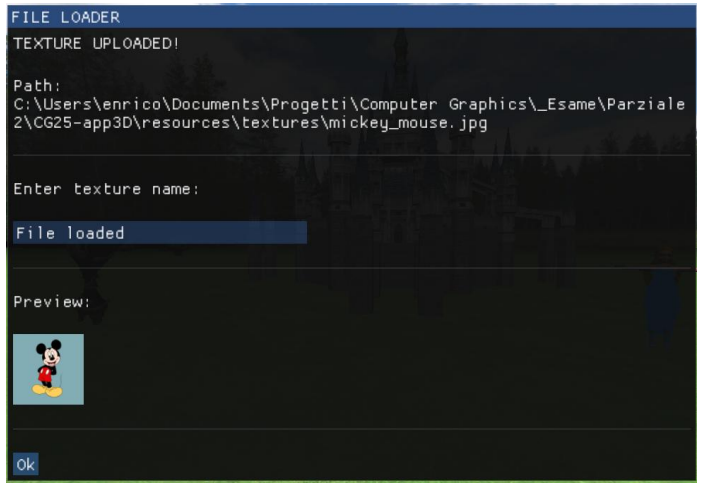
## FILE MESH

Caricando un file contenente delle mesh (.obj o .bda), in caso di successo verrà visualizzata la maschera per assegnare un nome all'oggetto e per modificarne la matrice di modellazione.

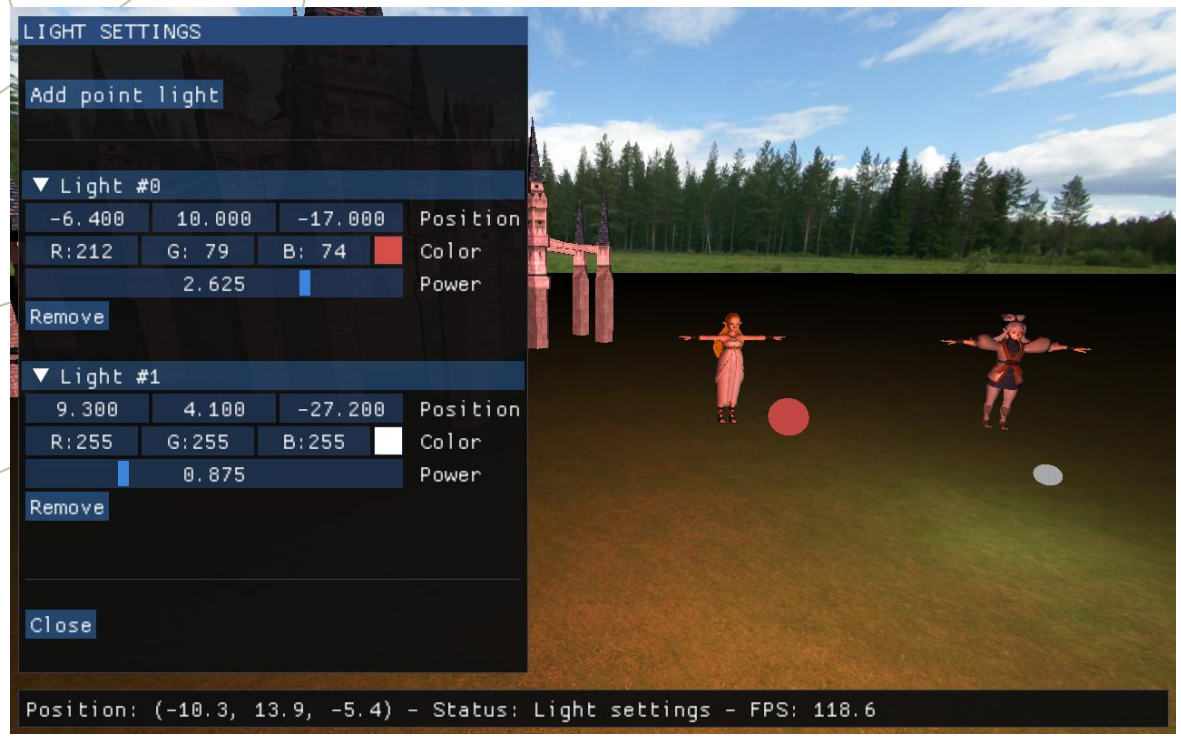



## FILE TEXTURE

Caricando un'immagine (.jpg, .jpeg o .png) verrà trattata come texture ed aggiunta alla lista delle texture dispoonibili da assegnare alle mesh.



# ILLUMINAZIONE

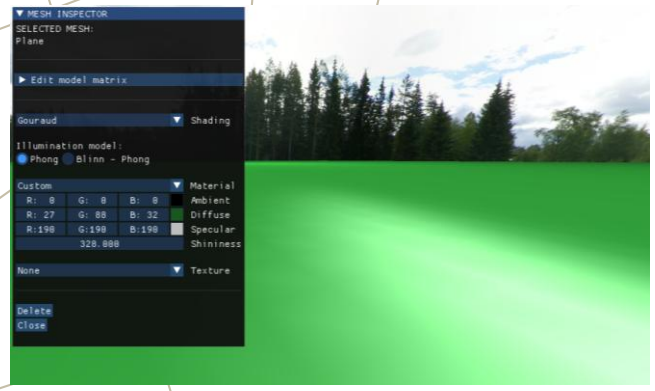



Cliccando il **pulsante L** si accede al menu di gestione delle luci, da cui è possibile aggiungere, modificare e rimuovere le **luci puntiformi**.

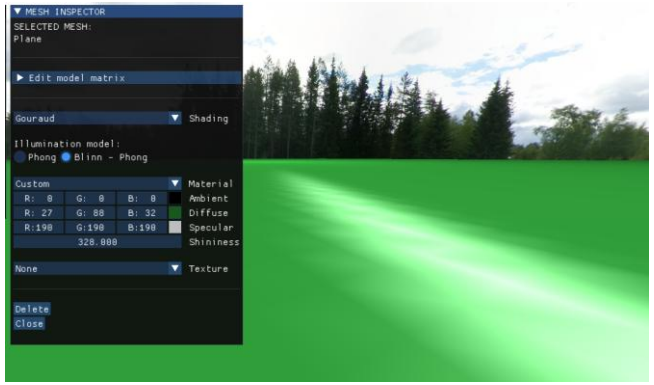
Di ogni luce si possono scegliere posizione, colore e potenza.

Per poter visualizzare le sorgenti, ogni luce è disegnata nella scena come una piccola **sfera**, colorata in base al colore della luce e alla luminosità impostata (potenze più basse corrispondono a un colore più grigio).

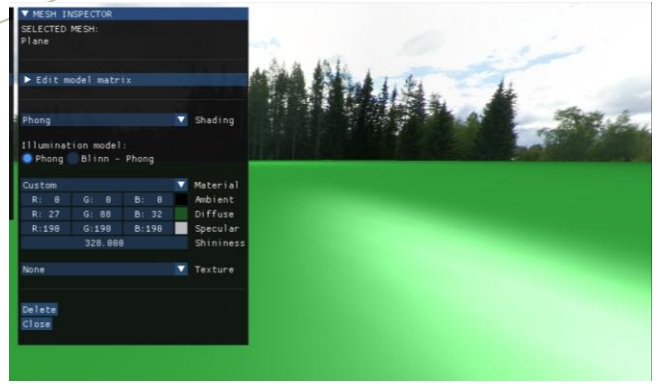
# ILLUMINAZIONE



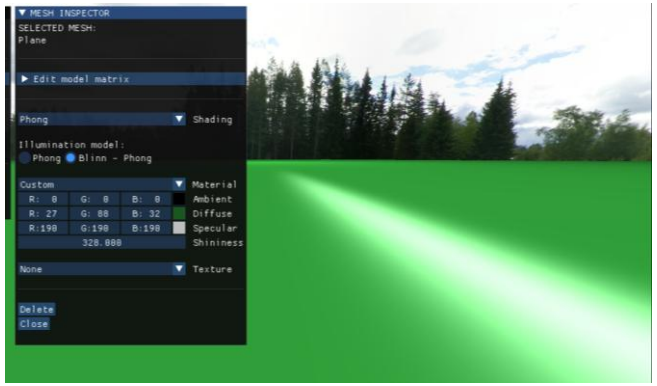

Gouraud - Phong




Gouraud – Blinn Phong




Phong - Phong




Phong – Blinn Phong

Per ogni mesh è possibile selezionare una **tecnica di shading**: in particolare per le tecniche di Gouraud e Phong sono stati implementati entrambi i **modelli di illuminazione** di Phong e Blinn-Phong.

Notiamo che:

- **Gouraud Shading:** Rimangono in evidenza le geometrie (vettori calcolati solo per i vertici)
- **Phong Shading**: L'illuminazione è più omogenea (vettori interpolati per ogni frammento)
- **Modello di Phong**: Riflessi quasi circolari
- **Modello di Blinn – Phong**: Riflessi ellittici più accurati per angoli di vista radenti

# COLLISIONI

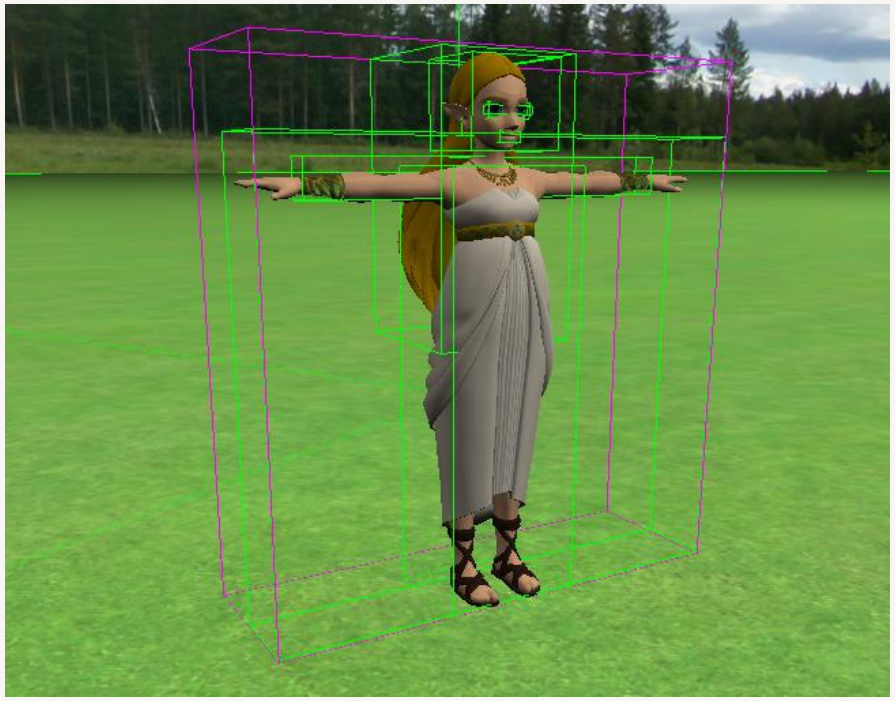
Ogni oggetto possiede un bounding box generale, aggiornato a ogni modifica di una sua mesh, stabilendo una **gerarchia a due livelli**.

Il sistema esegue l'algoritmo di collisione **Sfera – OBB** (Oriented Bounding Box) tra la telecamera e ciascun oggetto della scena:

- In assenza di collisione, il controllo procede all'oggetto successivo
- In caso di collisione, l'algoritmo viene rieseguito per il bounding box di ogni singola mesh dell'oggetto

Con la visualizzazione BoundingBox attiva, i volumi delle **mesh** sono tracciati in **verde**, mentre quelli **generici** dell'oggetto in **magenta**.

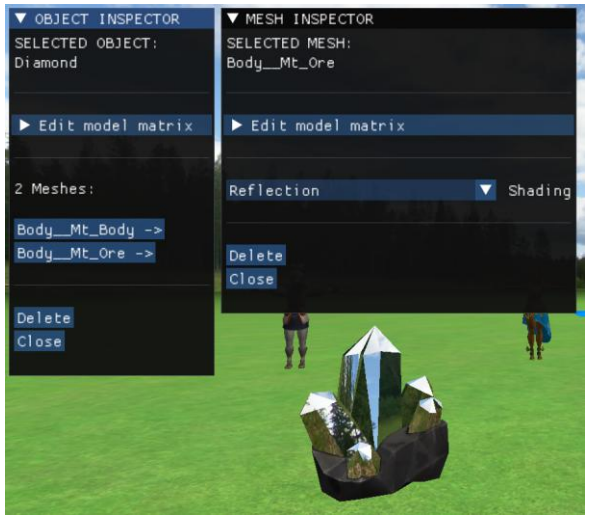
Le collisioni vengono controllate allo spostamento della telecamera, al movimento della trackball e durante la rotazione dell’oggetto animato.



# EFFETTI VISIVI

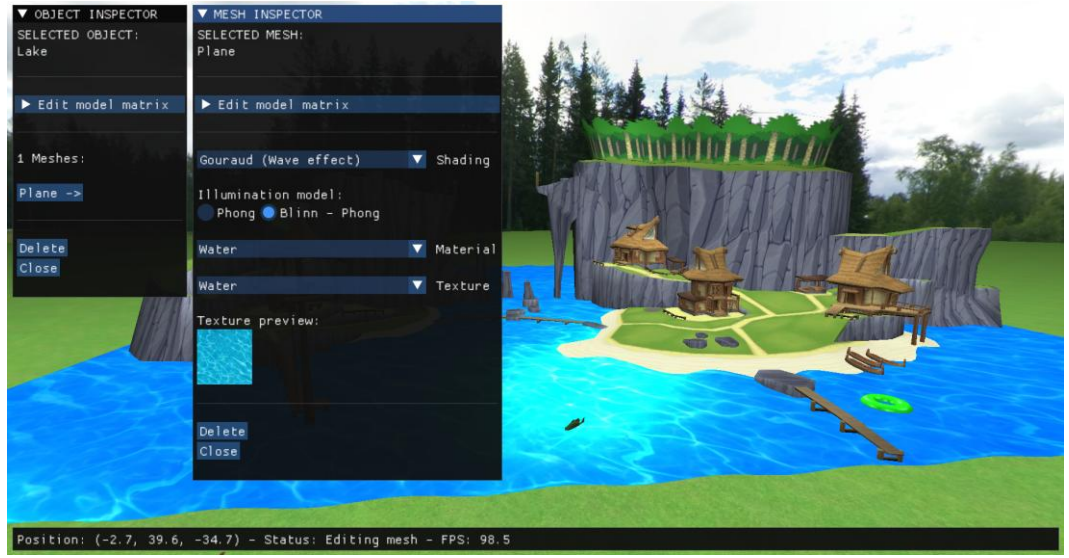
## RIFLESSIONE

La shader Reflection consente di riflettere la skybox sull’oggetto selezionato. Nella configurazione di esempio questa shader è applicata al diamante.

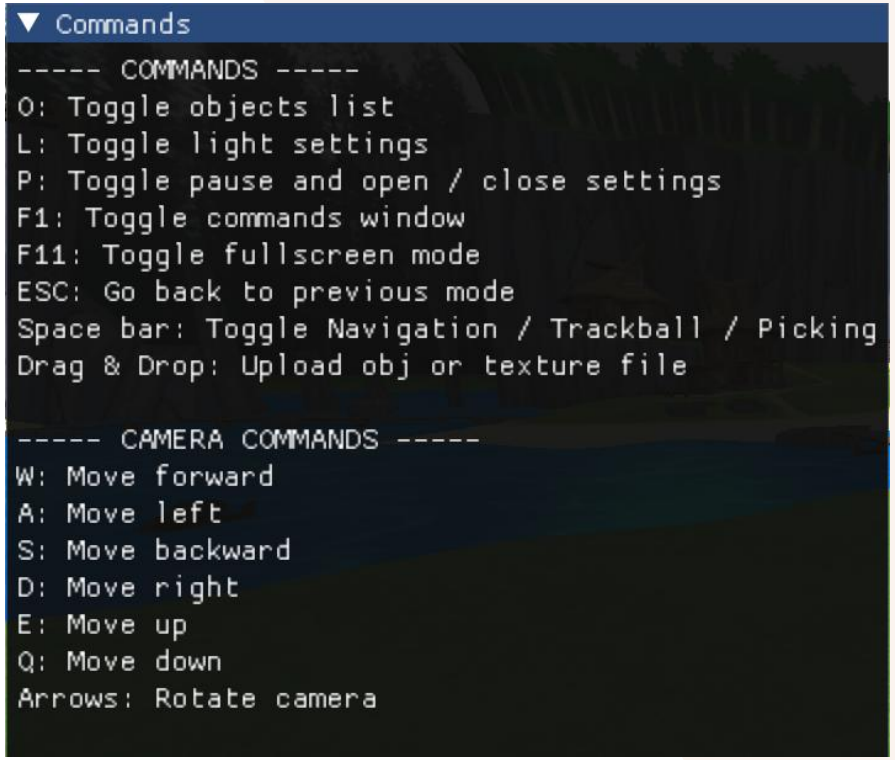



## ONDE

Per rendere animata la scena dell’isola, l’acqua è simulata da un piano con una texture, creando delle onde in movimento tramite apposita shader.



# COMANDI UTENTE




Schermata aperta alla pressione del tasto F1

# PROGETTAZIONE DEL SOFTWARE

## Sommario

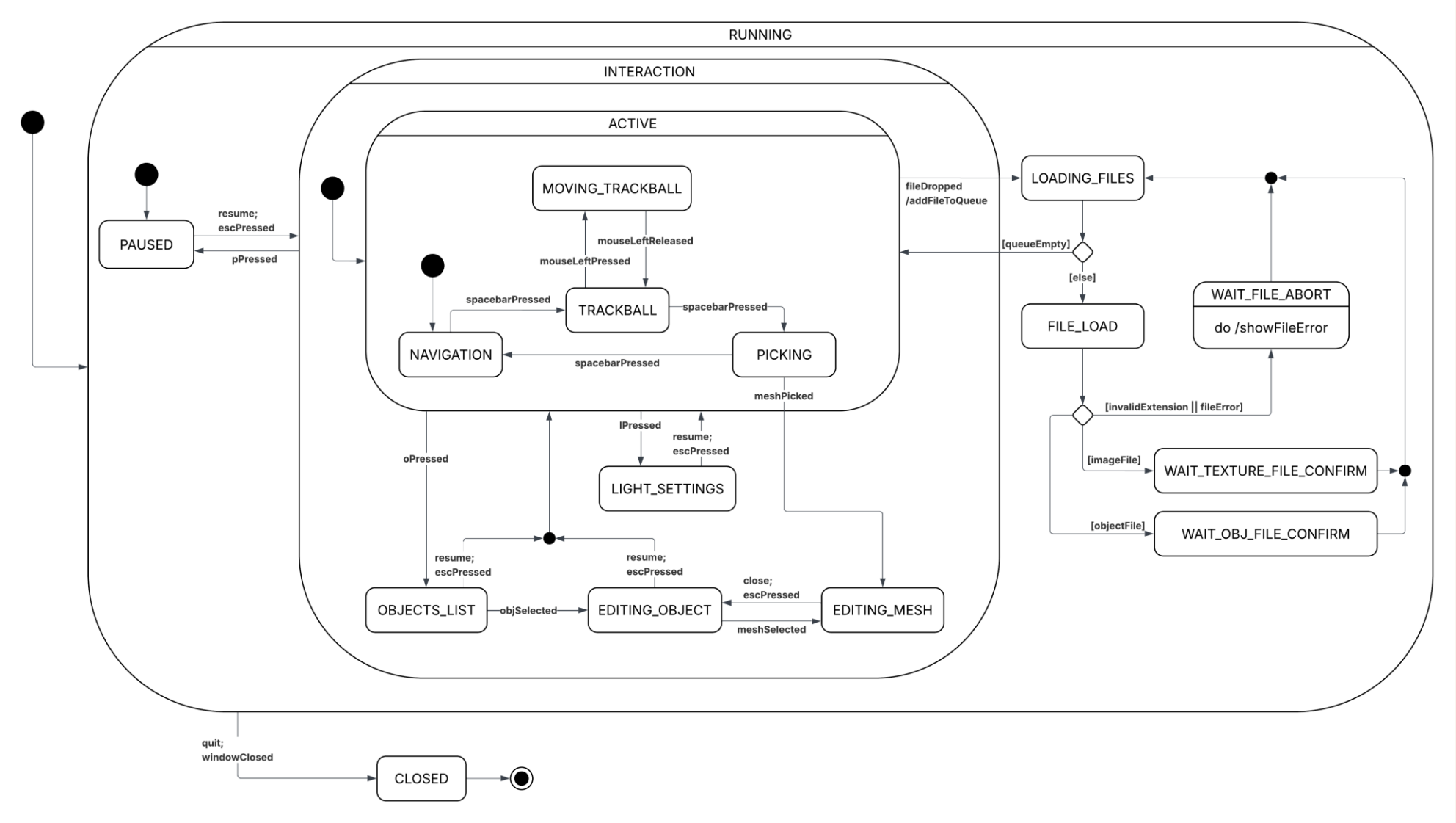
1. Diagramma a stati finiti
2. Architettura generale
3. Shading e Illuminazione

# DIAGRAMMA A STATI FINITI

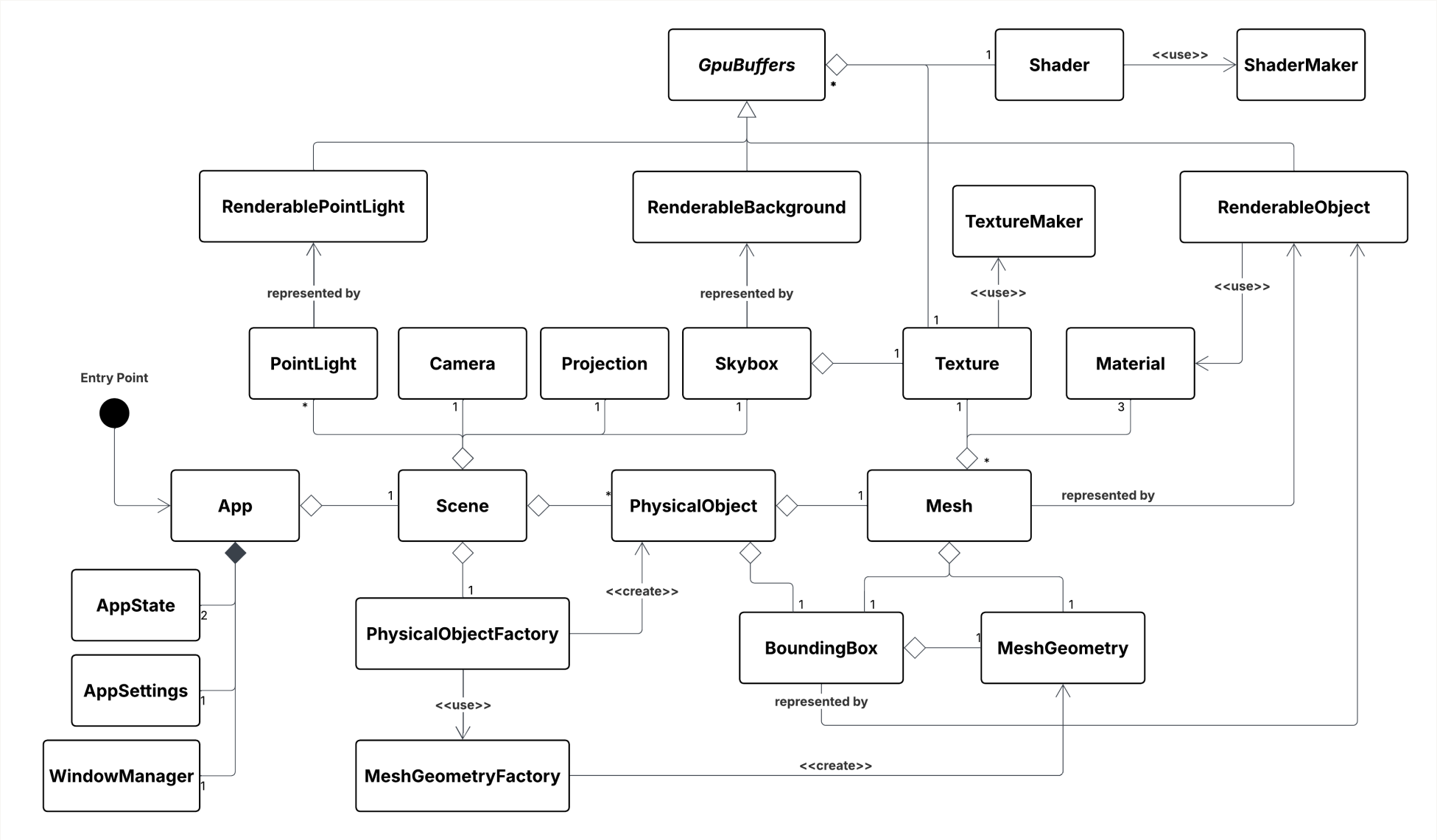
Per **differenziare gli input** in base allo stato del sistema e permettere l'esecuzione di alcune azioni solo in determinati momenti, è stata implementata una macchina a stati finiti.

Nel diagramma degli stati che si riporta succesivamente sono state inserite alcune **semplificazioni** per non appesantire troppo lo schema, ad esempio: utilizzando il pulsante ESC per tornare alla modalità di navigazione precedente, non si riparte dalla navigazione, ma viene ripristinata la modalità lasciata in precedenza.

# DIAGRAMMA A STATI FINITI



# ARCHITETTURA GENERALE



# SHADING E ILLUMINAZIONE

Vista la necessità di implementare i **modelli di illuminazione** Phong e Blinn-Phong con entrambe le **tecniche di Shading** Gouraud e Phong, ho scelto di:

- Realizzare un **unico file** per ogni tecnica di shading (diviso in fragment e vertex)
- Dentro allo shader, viene deciso se calcolare il **vettore riflessione** o **halfway** per la componente speculare a seconda dell'abilitazione del flag Blinn-Phong passato come uniform

```
// ----- SPECULAR COMPONENT -----
float cos_alfa;
// Changes the calculation of specular reflection of light based on the chosen lighting model
if (uUseBlinnPhong) {
    // Compute Halfway Vector (Vector halfway between the direction of view and the direction of light)
    vec3 H = normalize(L + V);
    cos_alfa = pow(max(dot(H, N), 0), material.shininess * 4);
} else {
    // Compute reflection direction R (light direction L compared to normal N)
    vec3 R = normalize(reflect(-L, N));
    cos_alfa = pow(max(dot(V, R), 0), material.shininess);
}
vec3 specular = lights[i].color * cos_alfa * material.specular;
```

gouraud.vert

```
// ----- SPECULAR COMPONENT -----
float cos_alfa;
// Changes the calculation of specular reflection of light based on the chosen lighting model
if (uUseBlinnPhong) {
    vec3 H = normalize(L[i] + V[i]);
    cos_alfa = pow(max(dot(H, N), 0), material.shininess * 4);
} else {
    cos_alfa = pow(max(dot(V[i], R[i]), 0), material.shininess);
}
vec3 specular = lights[i].color * cos_alfa * material.specular;
```

phong.frag

# GRAZIE

Bartocetti Enrico

enrico.bartocetti@studio.unibo.it

C.d.L. in Ingegneria e Scienze Informatiche

Corso di Computer Graphics

Prof. Damiana Lazzaro

A.A. 2025/2026